

Week 16 - Agentify the Assistant

Royas Shakya · AI Fellowship 2026 · Task 3

This submission extends the W15 RAG assistant with model-directed cross-source verification. A fixed pipeline is insufficient because the next source and search query depend on gaps or disagreements discovered in earlier results.

a. Context Engineering Technique

ContextManager caps each evidence item at 3,000 characters and compacts accumulated findings above 6,000 characters before the next decision. Summaries incorporate prior findings and request preservation of sources and disagreements; error notes are retained separately. Rebuilding the request from the original question and current findings replaces verbose history. Summary output is capped at 2,000 characters; summarizer failure uses truncation. This limits context growth, with the trade-off that detail can be lost. Identical successful tool calls reuse their result within a request.

b. Agentic Pattern

A single agent chooses knowledge search, web search, calculator/date tools, an answer, or clarification. Later searches depend on earlier evidence, so separate specialists and coordination overhead are not justified here. Compaction addresses context saturation; the same model checking its own work remains a self-verification limitation. Requests stop on answer, clarification, unrecovered error, or five decision iterations. Exhaustion reports incomplete verification. Clarification ends the request; the user resubmits with the missing information.

c. Evaluation Harness

The custom Python harness tests ten queries using expected stop states, required successful tools, flat argument schemas and lexical answer rubrics. It measures task completion, tool correctness, decision-trajectory length and tokens. Hard failures are execution failures; soft failures miss a rubric or trajectory criterion; cascading-soft candidates combine a tool failure, later tool actions and an unmet rubric. Human review must establish causality and citation support. JSON preserves answers and trace excerpts. The results report separates real model evaluation from controlled regression tests.

Additional Requirements

1. Skill vs. Agent. A Skill could describe the verification procedure, but requires an execution host to retrieve data and branch on results. This project supplies that host as a bounded loop and exposes the existing retriever through `search_knowledge`. `answer` and `ask_user` are terminal control actions, not additional agents.
2. Token and Cost Accounting. Totals include every successful decision and compaction response; local generation uses the model tokenizer and Gemini counts reported thinking tokens. Failed calls can have unreported consumption. Offline tokens are synthetic. A multi-agent baseline is not applicable; no unmeasured dollar cost or overhead multiplier is claimed.
3. Failure Injection. Controlled tests inject search exceptions and provider timeouts into the actual loop, check error propagation, exclude failed sources and verify a configured fallback. A separate live injection runner tests how the model responds to unavailable web search and malformed retrieval. The results report identifies which tests ran and their outcomes; scripted behavior is not presented as autonomous live recovery.
4. Tool vs. Agent Boundary. Web search is a bounded DDGS request with at most five results and a ten-second client timeout. Although the service may consult multiple engines internally, the assistant owns the research state and next decision. There is no delegated autonomous goal or agent-to-agent conversation, so this is a tool boundary.